Dynamo: Amazon's Highly Available Key-value Store

Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati,
Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall
and Werner Vogels

Amazon.com

Overview

- Amazon runs a world-wide e-commerce platform that serves **tens of millions customer**s at peak times using **tens of thousands of servers** located in **many data centers** around the world.
- There are **strict operational requirements** on Amazon's platform in terms of performance, reliability and efficiency, and to support continuous growth the platform needs to be highly scalable.
- Reliability at massive scale is one of the biggest challenges - even the slightest outage has significant financial consequences and impacts customer trust.
- This paper presents the design and implementation of Dynamo, a highly available key-value storage system that some of Amazon's core services use to **provide an "always-on" experience.**
- To achieve this level of availability, Dynamo **sacrifices consistency** under certain failure scenarios.

Overview

- Amazon uses a highly decentralized, loosely coupled, service oriented architecture consisting of hundreds of services.
- In this environment there is a particular need for **storage technologies** that are **always available**
  - For example, customers **should be able to view and add items to their shopping cart** even if disks are failing, network routes are flapping, or data centers are being destroyed by tornados.
  - The service responsible for managing shopping carts requires that it can **always write to and read from its data store**, and that its data needs to be available **across multiple data centers.**

Overview

- For example, the service that maintains shopping cart (Shopping Cart Service) served **tens of millions requests** that resulted in well over **3 million checkouts in a single day** and the service that manages session state handled **hundreds of thousands of concurrently active sessions.**

- A page request to one of the e-commerce sites typically requires the rendering engine to construct its response by **sending requests to over 150 services**.

- Example of a simple Service Level Agreement (SLA) is a service guaranteeing that it will provide a response within 300ms for 99.9% of its requests for a peak client load of 500 requests per second

Overview

The paper

- Shows how different techniques can be combined to provide a single highly-available system.
- Demonstrates that an eventually-consistent storage system can be used in production with demanding applications.
- Provides insight into the tuning of these techniques to meet the requirements of production systems with very strict performance demands.
  - Example of a simple Service Level Agreement (SLA) is a service guaranteeing that it will provide a response within 300ms for 99.9% of its requests for a peak client load of 500 requests per second.

Overview

- For many services, such as those that provide best seller lists, shopping carts, customer preferences, session management, sales rank, and product catalog, the common pattern of **using a relational database would lead to inefficiencies** and limit scale and availability.
- No operations **span multiple data items** and there is no need for relational schema.
- Dynamo provides a simple **primary-key only interface** to meet the requirements of these applications.

Overview

Limitation of relation databases

- A relational database is a solution that is far from ideal. Most of these services only store and **retrieve data by primary key** and do not require the complex querying and management functionality offered by a traditional RDBMS (e.g., MySQL).
- In addition, the **available replication technologies** are limited and typically choose **consistency over availability.**
- Although many advances have been made in the recent years, it is still not easy to scale-out databases or use smart partitioning schemes for load balancing.

Consistency in replicated data stores

- Consistency model
  - The **clauses** in a **contract** between data store and processes/clients.

- Consistency protocol
  - How to **implement** the clauses in the contract the data store and processes/clients agreed up on.

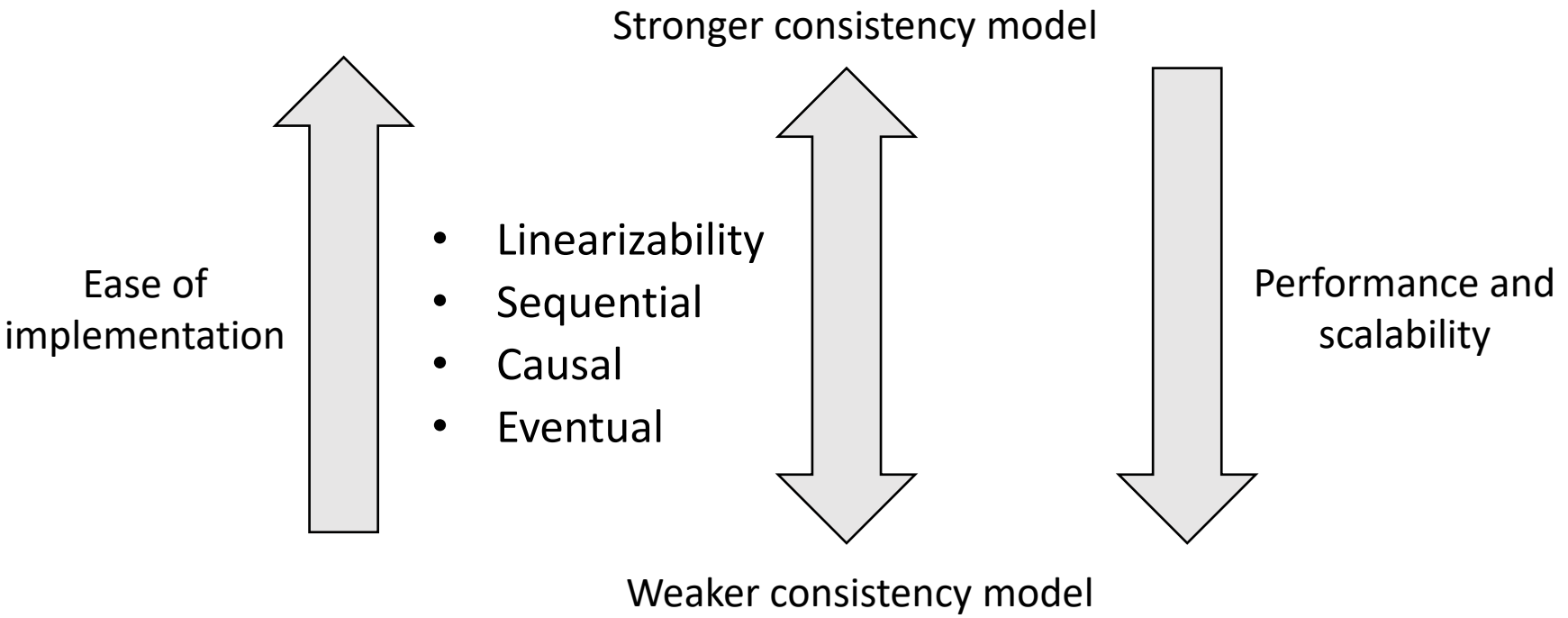
Consistency in replicated data stores

- Data-centric consistency model
  - From the perspective of all processes and/or clients
  - Concurrent processes/clients may be simultaneously updating shared data
  - Constraints on **operation ordering at the data-store level**

- Client-centric consistency model
  - From the perspective of a single client
  - Constraints on **operation ordering for each client independently**
  - Provides guarantees for a single client concerning the consistency of accesses to a data store by that client
  - No guarantees are given concerning concurrent accesses by different clients (i.e., the behavior is undefined, not part of the contract)

Consistency models

- Consistency models are **different ways to define the correct behavior** of operations.
- In a distributed system, often read/write operations are concurrent and the order in which they occur is nondeterministic.
- Guaranteeing global ordering on concurrent and conflicting operations may be a **costly task**, downgrading performance and scalability.
- Possible solution: weaken consistency requirements so that hopefully global synchronization can be avoided.



CAP theorem

The CAP theorem states that in a distributed system (e.g., a distributed data store) it is impossible to attain all three simultaneously.

Consistency

Availability

Partition-tolerance

To be exact, it is **not** any 2-out-of-the-3. In the event the network is partitioned (P), the system must choose either consistency (C) or availability (A) and cannot provide both simultaneously. So, **CP** or **AP** and not both.

CP system: In the event of a partition, choose **C** over **A**, e.g., distributed data stores that demand ACID (Atomicity, Consistency, Isolation, and Durability) properties.

AP system: In the event of a partition, choose **A** over **C**, e.g., eventually consistent distributed data stores where clients remain operational in the presence of stale data.

Overview

In Dynamo,

- Data is partitioned and replicated using consistent hashing, and consistency is facilitated by object versioning.
- The consistency among replicas during updates is maintained by a quorum-like technique and a decentralized replica synchronization protocol.
- Dynamo employs a gossip based distributed failure detection and membership protocol.
- Storage nodes can be added and removed from Dynamo without requiring any manual partitioning or redistribution.

- The tradeoffs are in performance, cost efficiency, availability, and durability guarantees.

Summary of techniques used in Dynamo

Problem	Technique	Advantage
Partitioning and request routing	Consistent hashing	Incremental scalability, load balancing etc.
High availability for writes	- Eventual consistency - Reconciliation during reads (uses vector clocks) - Quorum protocol	Version size is decoupled from update rates
Handling temporary failures	"Sloppy" Quorum protocol and hinted handoff	Provides high availability and durability guarantee when some of the replicas are not available
Recovering from permanent failures	Anti-entropy using Merkle trees	Synchronizes divergent nodes in the background
Membership and failure detection	Epidemic/gossip-based membership protocol	Preserves symmetry and avoids having a centralized registry for storing membership and node liveness information.

Design Considerations

- To achieve strong consistency, many systems are forced to tradeoff the availability of the data under certain failure scenarios.
- When dealing with the possibility of network failures, strong consistency and high data availability **cannot be achieved simultaneously**.

- Dynamo is designed to be an **eventually consistent** data store; that is all updates reach all replicas eventually.

- The **challenge** with this approach is that it can lead to **conflicting changes** which must be detected and resolved.
- An important design consideration is to decide ***when*** to perform the process of **resolving update conflicts**, i.e., whether conflicts should be **resolved during reads or writes**.

Design Considerations

- Dynamo targets the design space of an “always writeable” data store (i.e., a data store that is **highly available for writes**). **No updates are rejected** due to failures or concurrent writes.
- For a number of Amazon services, **rejecting customer updates** could result in a **poor customer experience**.
  - For instance, the shopping cart service must allow customers to add and remove items from their shopping cart even amidst network and server failures.
- This requirement forces us to **push the complexity of conflict resolution to the reads** in order to ensure that writes are never rejected.

Design Considerations

- Dynamo can be characterized as a zero-hop DHT, where each node maintains **enough routing information locally** to route a request to the appropriate node directly. Key lookup cost is O(1).

- Dynamo is built for **latency sensitive applications** that require at least 99.9% of read and write operations to be performed within **a few hundred milliseconds**.

- To meet these stringent latency requirements, it was imperative for us to **avoid routing requests through multiple nodes** (which is the typical design adopted by several distributed hash table systems such as Chord and Pastry).

Design Considerations

Other key principles embraced in the design are

- **Incremental scalability**: Dynamo should be able to scale out one storage host (node) at a time, with minimal impact on both operators of the system and the system itself.
- **Symmetry**: Every node in Dynamo should have the same set of responsibilities as its peers; there should be no distinguished node or nodes that take special roles or extra set of responsibilities.
  - An extension of symmetry, the design should favor **decentralized** peer-to-peer techniques over centralized control. This leads to a simpler, more scalable, and more available system.
- **Heterogeneity**: The system needs to be able to exploit heterogeneity in the infrastructure it runs on. e.g. the **work distribution must be proportional** to the capabilities of the individual servers. This is essential in adding new nodes with higher capacity without having to upgrade all hosts at once.

System Architecture

- **System Interface**
- Get(key) returns a single object or a list of objects with conflicting versions along with a *context*.
- Dynamo applies a MD5 hash on the key to generate a 128-bit identifier.

System Architecture

- **Partitioning Algorithm**
- Dynamo's partitioning scheme relies on consistent hashing to distribute the load across multiple storage hosts.
- The basic consistent hashing algorithm presents some challenges.
  - First, the random position assignment of each node on the ring leads to non-uniform data and load distribution.
  - Second, the basic algorithm is oblivious to the heterogeneity in the performance of nodes.
- Instead of mapping a node to a single point in the circle, each node gets assigned to multiple points in the ring. To this end, Dynamo uses the concept of "virtual nodes".
- A virtual node looks like a single node in the system, but **each node can be responsible for more than one virtual node**. Effectively, when a new node is added to the system, it is assigned **multiple positions** (henceforth, "tokens") in the ring.

System Architecture

- **Partitioning Algorithm**
- A newly available node accepts a roughly equivalent amount of load from each of the other available nodes.
- The number of virtual nodes that a node is responsible can decided based on its capacity, accounting for heterogeneity in the physical infrastructure.

System Architecture

- **Replication**
- To achieve high **availability and durability**, Dynamo replicates its data on multiple hosts.
- Each data item is replicated at N hosts, where N is a parameter configured *“per-instance”*.
- The **coordinator** is in charge of the replication of the data items that fall within its range.
- In addition to locally storing each key within its range, the coordinator **replicates these keys at the N-1 clockwise successor nodes** in the ring.
- Each node is responsible for the region of the ring **between it and its Nth predecessor**.

System Architecture

- **Replication**
- Node B replicates the key *k* at nodes C and D in addition to storing it locally.
- Node D will store the keys that fall in the ranges (A, B], (B, C], and (C, D].
- The **list of nodes** that is responsible for **storing a particular key** is called the ***preference list***.
- Every node in the system can determine which nodes should be in this list for any particular key.
- To account for node failures, **preference list contains more than N nodes**.
- The preference list for a key is constructed by **skipping positions in the ring** to ensure that the list contains only **distinct physical nodes**.

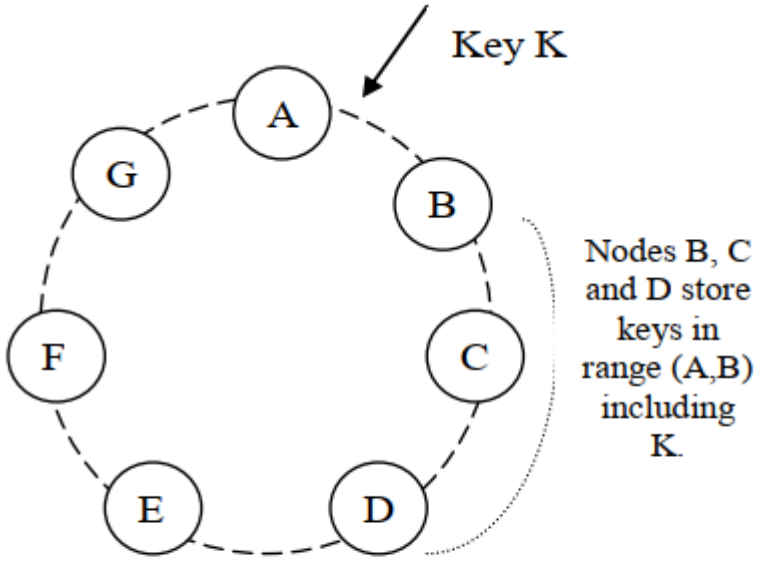


Figure 2: Partitioning and replication of keys in Dynamo ring.

System Architecture

- **Data versioning**
- Dynamo provides **eventual consistency**, which allows for updates to be propagated to all replicas asynchronously.
- A put() call may **return to its caller before the update has been applied at all the replicas**, which can result in scenarios where a subsequent get() operation may return an object that **does not have the latest updates**.
- Under certain failure scenarios (e.g., server outages or network partitions), updates may not arrive at all replicas for an extended period of time.
- For example, the shopping cart application requires that an “Add to Cart” operation can **never be forgotten or rejected**. If the most recent state of the cart is unavailable, and a user makes changes to an older version of the cart, that change is still meaningful and should be preserved. But at the same time it shouldn’t supersede the currently unavailable state of the cart, which itself may contain changes that should be preserved. Note that both “*add to cart*” and “*delete item from cart*” operations are translated into put requests to Dynamo. **Divergent versions are reconciled later**.

System Architecture

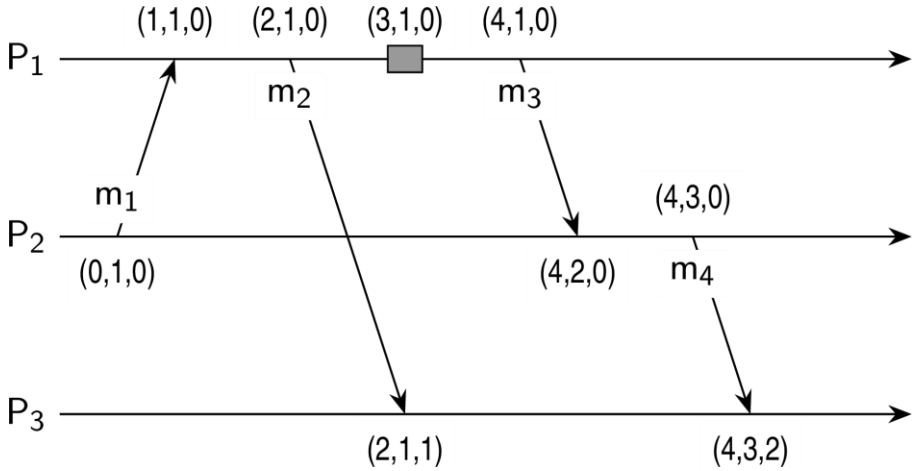
- **Data versioning**
- In order to provide this kind of guarantee, Dynamo treats the result of each modification as **a new and immutable version of the data**. It allows for multiple versions of an object to be present in the system at the same time.
- Version branching may happen, in the presence of failures combined with concurrent updates, resulting in conflicting versions of an object.
- The system cannot reconcile the multiple versions of the same object and the **client must perform the reconciliation** in order to *collapse* multiple branches of data evolution back into one (semantic reconciliation).
- A typical example of a collapse operation is “merging” different versions of a customer’s shopping cart.
  - Using this reconciliation mechanism, an “add to cart” operation is never lost.
  - However, deleted items can resurface.

System Architecture

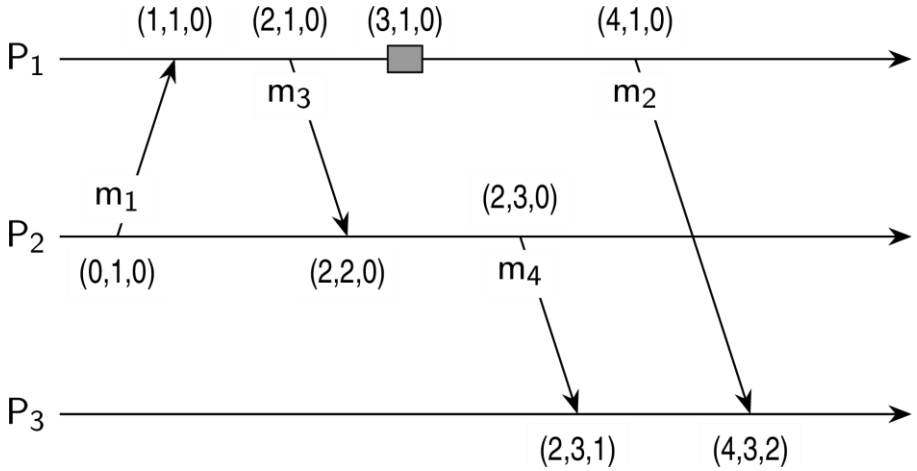
- **Data versioning**
- Dynamo uses **vector clocks** in order to capture causality between different versions of the same object.
- A vector clock is effectively a list of (node, counter) pairs. One vector clock is associated with every version of every object.

Vector clocks: example

Capturing potential causality when exchanging messages



(a)



(b)

Analysis

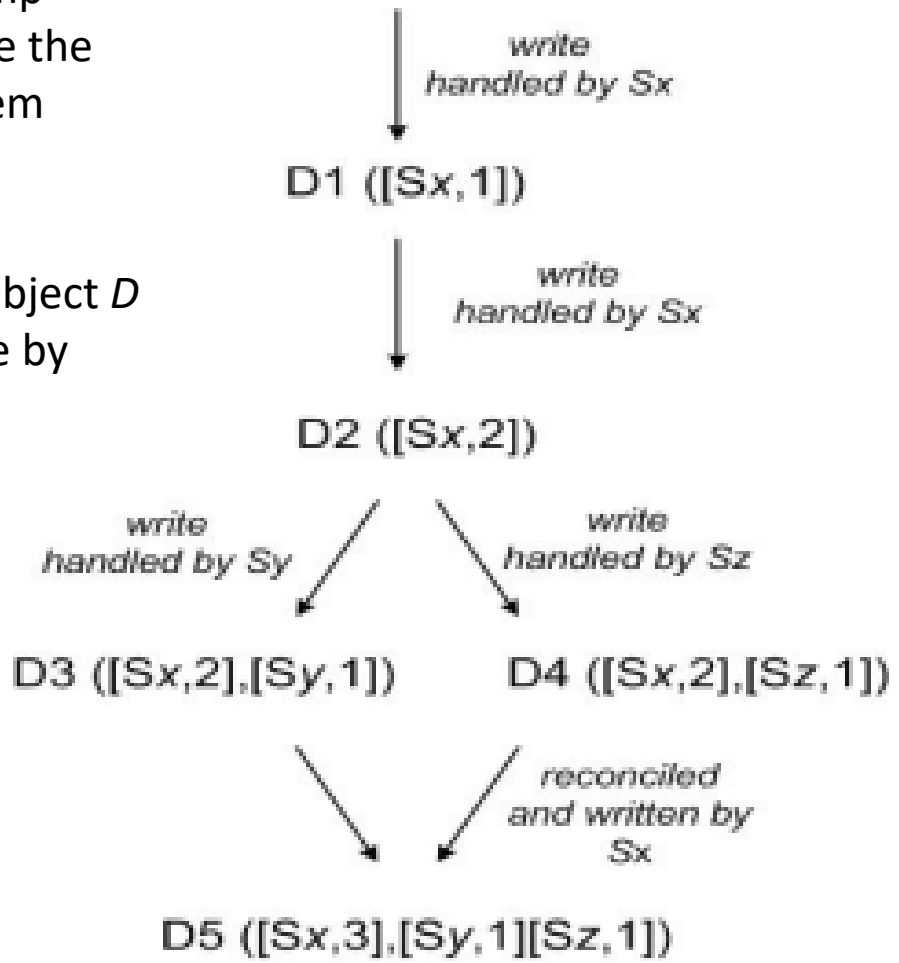
Situation	$ts(m_2)$	$ts(m_4)$	$ts(m_2) < ts(m_4)$	$ts(m_2) > ts(m_4)$	**Conclusion**
(a)	(2, 1, 0)	(4, 3, 0)	Yes	No	m_2 may causally precede m_4
(b)	(4, 1, 0)	(2, 3, 0)	No	No	m_2 and m_4 may conflict

Object Versioning and Reconciliation

Dynamo stores a timestamp that indicates the last time the node updated the data item

E.g., *D3* ([*Sx*, 2], [*Sy*, 1]): Object *D* version 3 has written once by server *Sx* and server *Sy*

Each node keeps all versions until the data becomes consistent



Divergent versions created because of network partitions

System Architecture

- **Execution of get () and put ()**
- A **node** handling a **read or write** operation is known as the ***coordinator***. Typically, this is the first among the **top N nodes** in the **preference list**.
- A node routes requests **directly** to the appropriate coordinator nodes. A get request is forwarded to the first among the top N nodes in the preference list.
- Read and write operations involve the first N healthy nodes in the preference list. When there are node failures or network partitions, nodes that are lower ranked in the preference list are accessed.

System Architecture

- **Execution of get () and put ()**
- Dynamo uses a consistency protocol similar to those used in **quorum systems**.
- This protocol has two configurable values: R and W. R is the minimum number of nodes that must participate in a successful read operation. W is the minimum number of nodes that must participate in a successful write operation.
- Setting R and W such that R + W > N yields a quorum-like system.
- The **latency** of a get (or put) operation is dictated by the **slowest** of the R (or W) replicas. For this reason, R and W are usually configured to be less than N, to provide better latency.
- If the coordinator ends up gathering multiple versions of the data, it returns all the versions it deems to be **causally unrelated**.
- The divergent versions are then reconciled and the reconciled version superseding the current versions is written back.

System Architecture

- **Execution of get () and put ()**
- Write requests are coordinated by a node in the key's current preference list.
- This is due to the fact that these preferred nodes have the added responsibility of creating a new version stamp.

System Architecture

- **Handling Failures: Hinted Handoff**
- If Dynamo used a traditional quorum approach **it would be unavailable during server failures and network partitions**, and would have reduced durability.
- Dynamo does not enforce strict quorum membership and instead it uses a "**sloppy quorum**"; all read and write operations are performed on the **first N *healthy* nodes from the preference list**, which may not always be the first N nodes encountered while walking the consistent hashing ring.

System Architecture

- **Handling Failures: Hinted Handoff**
- If node A is temporarily down or unreachable during a write operation then a replica that would normally have lived on A will now be sent to node D. This is done to maintain the **desired availability** and **durability** guarantees.
- The replica sent to D will have **a hint in its metadata** that suggests which node was the intended recipient of the replica (in this case A).
- Nodes that receive **hinted replicas** will keep them in a separate local database that is scanned periodically.
- Upon detecting that A has recovered, D will attempt to deliver the replica to A.
- Once the transfer succeeds, D may delete the object from its local store without decreasing the total number of replicas in the system.

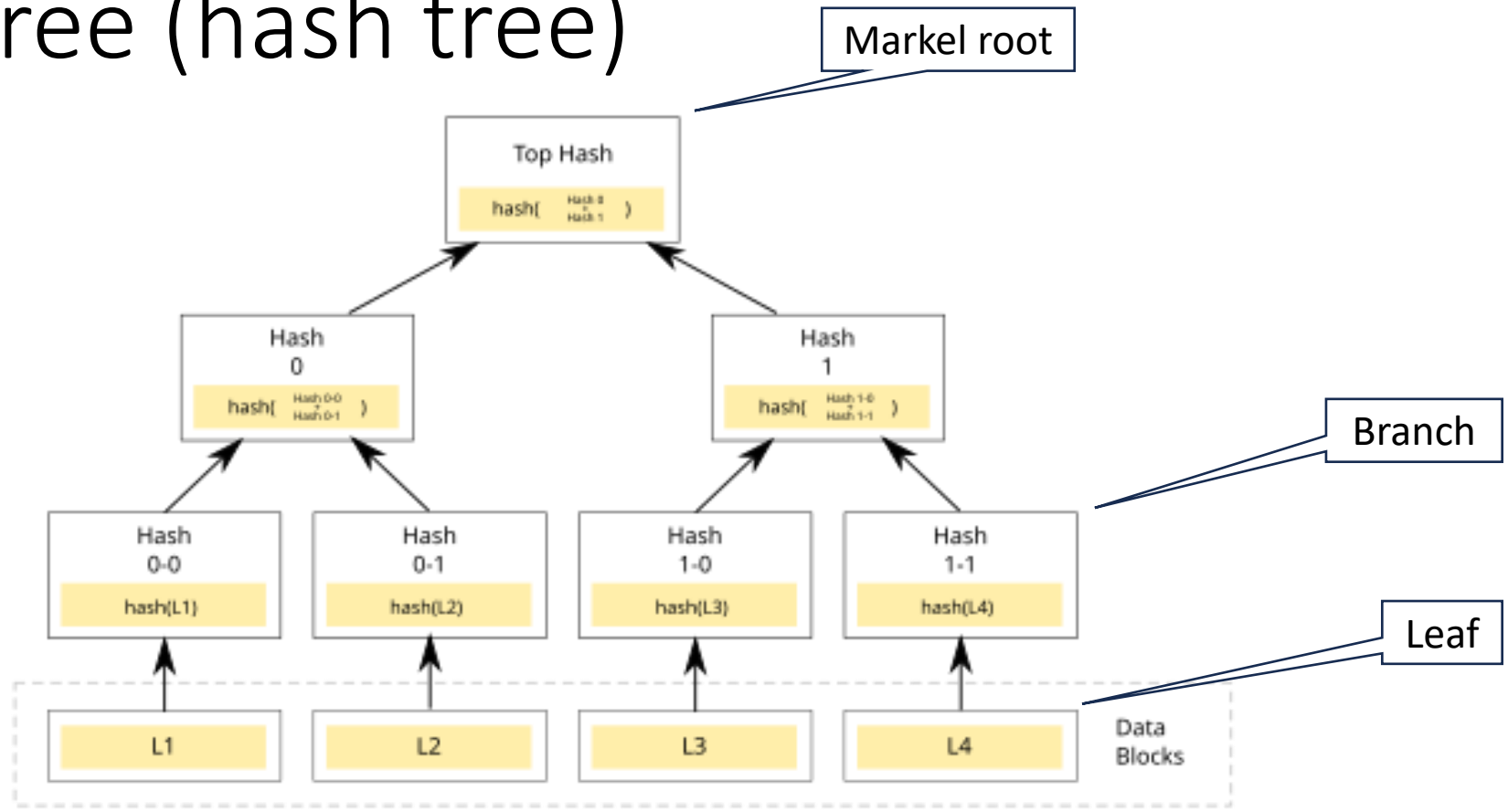
System Architecture

- **Handling Failures: Hinted Handoff**
- Using **hinted handoff**, Dynamo ensures that the read and write operations are **not failed due to temporary** node or network **failures**.
- Dynamo is configured such that each object is replicated across multiple data centers. In essence, the **preference list of a key** is constructed such that the **storage nodes** are spread across **multiple data centers**.

System Architecture

- **Handling permanent failures: Replica synchronization**
- Hinted handoff works best if the system membership churn is low and node failures are transient.
- There are scenarios under which **hinted replicas become unavailable** before they can be returned to the original replica node.
- Dynamo implements an **anti-entropy** (replica synchronization) protocol to keep the replicas synchronized.
- Each node maintains a separate **Merkle tree for each key range** (the set of keys covered by a virtual node) it hosts.
- The principal advantage of Merkle tree is that each branch of the tree can be checked independently without requiring nodes to download the entire tree or the entire data set.
- Moreover, Merkle trees help in reducing the amount of data that needs to be transferred while checking for inconsistencies among replicas.
- For instance, if the hash values of the **root of two trees** are equal, then the values of the leaf nodes in the tree are equal and the nodes require no synchronization. If not, it implies that the values of some replicas are different.
- In such cases, the nodes may exchange the hash values of children and the process continues until it reaches the leaves of the trees, at which point the hosts can **identify the keys that are "out of sync"**.

Merkle tree (hash tree)



An example of a binary hash tree. Hashes 0-0 and 0-1 are the hash values of data blocks L1 and L2, respectively, and hash 0 is the hash of the concatenation of hashes 0-0 and 0-1.

https://en.wikipedia.org/wiki/Merkle_tree

Merkle tree (hash tree)

A Merkle proof is a cryptographic method used to verify that a specific piece of data exists within a Merkle tree. Verification size grows logarithmically rather than linearly.

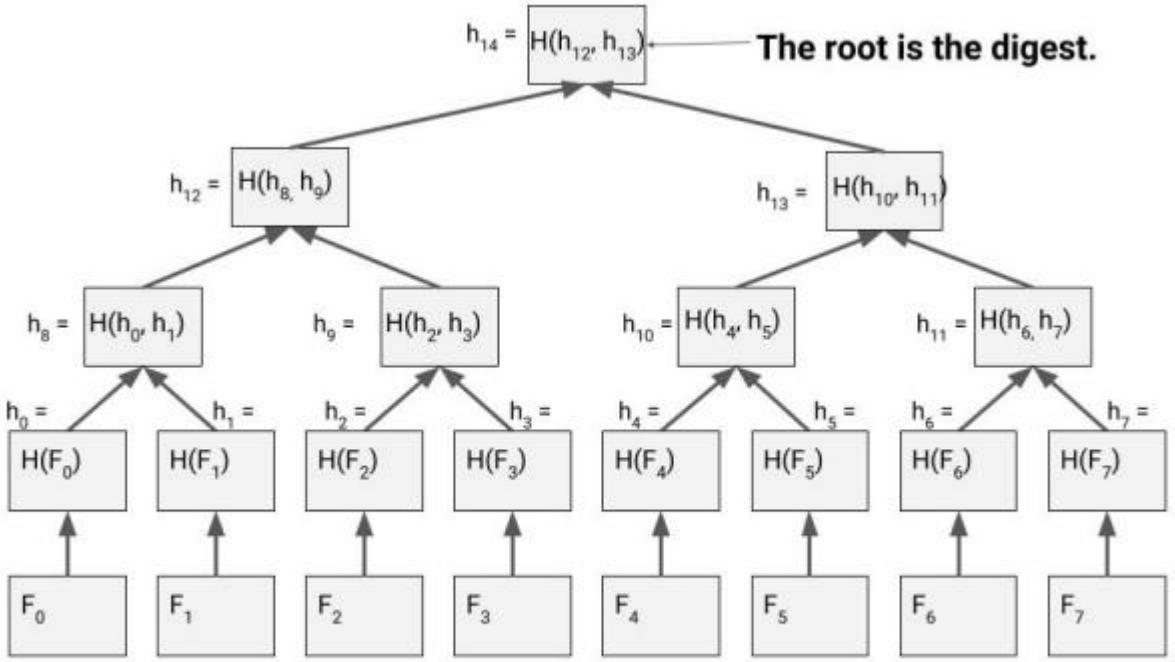


Figure 1: A Binary Merkle Tree

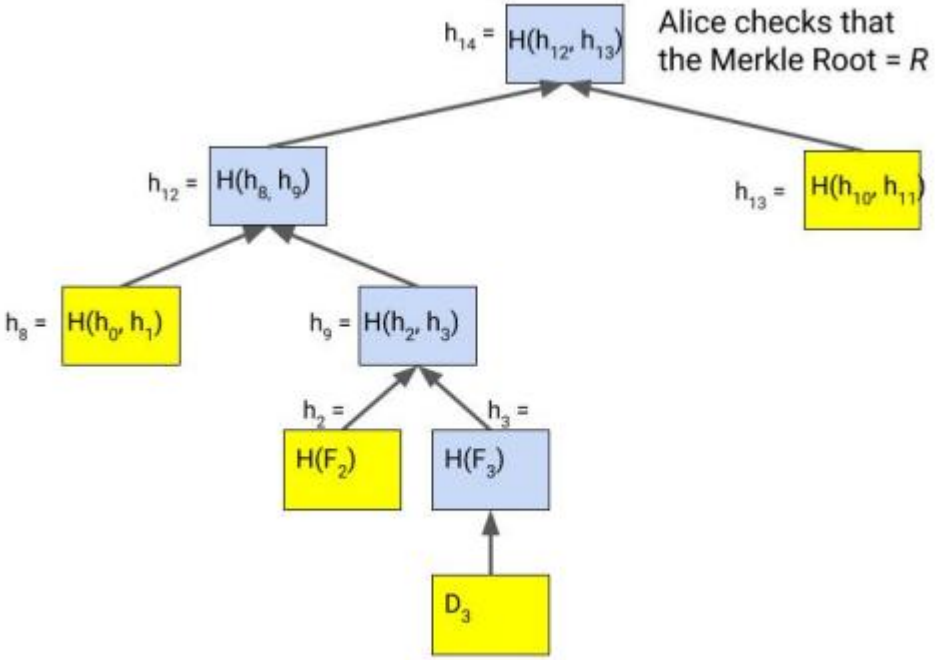


Figure 2: The Merkle Proof (in yellow)

The Merkle proof of a leaf in a Merkle tree consists of the siblings of each of the nodes in the path from the leaf to the Merkle root.

https://math.mit.edu/research/highschool/primes/materials/2018/Kuszmaul.pdf

Gossip-based coordination, data dissemination (also called epidemic protocol)

Paper: Márk Jelasity, Spyros Voulgaris, Rachid Guerraoui, Anne-Marie Kermarrec, and Maarten van Steen. 2007. Gossip-based peer sampling. ACM Trans. Comput. Syst. 25, 3 (August 2007)

The goal is to spread information in a large-scale distributed system, often using node-local information only.

Here, the key idea is that there is no central coordinator, initially nodes have no knowledge of the network topology. Nodes interact in a peer-to-peer manner.

Anti-entropy message propagation model:

- Nodes exchange information in rounds. In each round, a node *P* selects another node *Q* from the system at random. (Variations include *P* selecting multiple random nodes, *P* ignoring previously selected nodes.)
- Pull: *P* only pulls in new updates from *Q.*
- Push: *P* only pushes its own updates to *Q.*
- Push-pull: *P* and *Q* send updates to each other. For push-pull, it takes $O(log(N))$ rounds to disseminate updates to all *N* nodes (round = when every node has taken the initiative to start an exchange).

Application examples of gossip protocol:

a) Aggregating information, b) peer-sampling, c) overlay constriction, and d) Apache Cassandra distributed DB.

System Architecture

- **Membership (coordination) and Failure Detection**
- An **epidemic/gossip-based protocol** propagates membership changes and maintains an **eventually consistent** view of membership.
- Each node contacts **a peer chosen at random** every second and the two nodes efficiently reconcile their persisted membership change histories.

System Architecture

- **Membership (coordination) and Failure Detection**
- The mechanism described above could temporarily result in a **logically partitioned** Dynamo ring.
- To prevent logical partitions, some Dynamo nodes play the role of **seeds**.
- Seeds are nodes that are discovered via an external mechanism and are known to all nodes.
- Because all nodes eventually reconcile their membership with a seed, logical partitions are highly unlikely.

System Architecture

- **Failure Detection**
- For the purpose of avoiding failed attempts at communication, a purely **local notion of failure detection** is entirely sufficient:
  - Node A may consider node B failed if node **B does not respond to node A's messages** (even if B is responsive to node C's messages).
- A node A quickly discovers that a node B is unresponsive when B fails to respond to a message.
- Node A then uses alternate nodes to service requests that map to B's partitions.
- A periodically retries B to check for the latter's recovery.

System Architecture

- **Adding/Removing Storage Nodes**
- Based on the consistent hashing algorithm.

Implementation

- Each storage node has three main software components: request coordination, membership and failure detection, and a local persistence engine. All these components are implemented in Java.
- Dynamo’s local persistence component allows for different storage engines to be plugged in. Engines that are in use are Berkeley Database (BDB) Transactional Data Store2, BDB Java Edition, MySQL, and **an in-memory buffer** with persistent backing store.
- The main reason for designing a **pluggable persistence component** is to choose the storage engine best suited for an application’s access patterns.

Implementation

- **Each client request** results in the **creation of a state machine** (think 2-phase commit) on the node that received the client request.
  - The state machine contains all the logic for identifying the nodes responsible for a key, sending the requests, waiting for responses, potentially doing retries, processing the replies and packaging the response to the client.
- After the **read** response has been returned to the caller, the state machine waits for a small period of time to receive any outstanding responses.
- If stale versions were returned in any of the responses, the coordinator updates those nodes with the latest version.
- This process is called ***read repair*** because it repairs replicas that have missed a recent update at an opportunistic time and relieves the anti-entropy protocol from having to do it.

Implementation

- **Write** requests are coordinated by one of the top N nodes in the preference list.
- Any of the top N nodes in the preference list is allowed to **coordinate** the writes.
- In particular, since each write usually follows a read operation, the coordinator for a write is chosen to be the node that replied fastest to the previous read operation which is stored in the context information of the request.

Experiences and Lessons

- *Business logic specific reconciliation:*
- Dynamo is used by several services with different configurations.
- These instances differ by their version reconciliation logic, and read/write quorum characteristics.

- *High performance read engine:*
- While Dynamo is built to be an “always writeable” data store, a few services are tuning its quorum characteristics and using it as a high performance read engine.
- Typically, these services have a high read request rate and only a small number of updates.
- In this configuration, typically R is set to be 1 and W to be N.

- A typical value of N used by Dynamo’s users is 3. The values of W and R impact object availability, durability and consistency.

Experiences and Lessons

- If W is set to 1, then the system will never reject a write request as long as there is at least one node in the system that can successfully process a write request.
- However, low values of W and R can **increase the risk of inconsistency**.
- All the measurements presented in this paper were taken on a live system operating with a configuration of (3,2,2) and running a couple hundred nodes with homogenous hardware configurations.

Experiences and Lessons

- **Balancing Performance and Durability**
- Dynamo's principal design goal is to build a **highly available** data store, **performance** is an equally important criterion in Amazon's platform.
- A typical **SLA** required of services that use Dynamo is that 99.9% of the read and write requests execute within 300ms.
- Performance of these operations is limited by the slowest of the R or W replicas.

Experiences and Lessons

- Figure 4 shows the average and **99.9th percentile latencies** of Dynamo's **read and write operations** during a period of 30 days.
- The 99.9th percentile latencies are around 200 ms and are **an order of magnitude higher than the averages**.
- This is because the 99.9th percentile latencies are affected by several factors such as variability in request load, object sizes, and locality patterns.
- Dynamo provides the ability to **trade-off durability guarantees for performance**. In the optimization each storage node maintains an **object buffer in its main memory** (Figure 5). Each write operation is stored in the buffer and gets periodically written to storage by a *writer thread*.

Experiences and Lessons

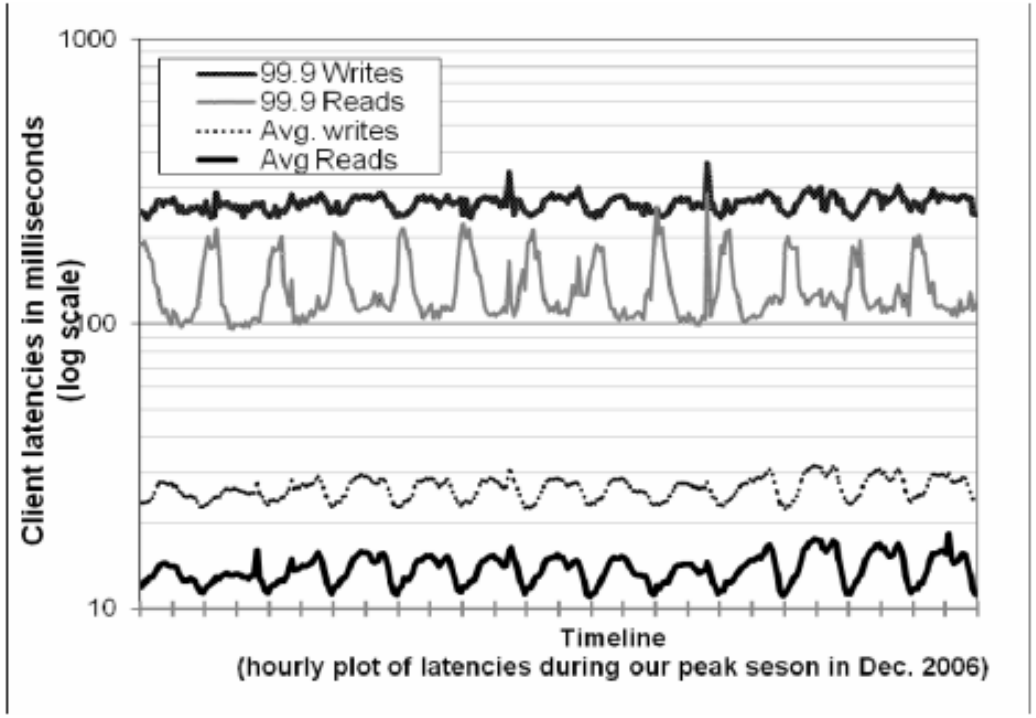


Figure 4: Average and 99.9 percentiles of latencies for read and write requests during our peak request season of December 2006. The intervals between consecutive ticks in the x-axis correspond to 12 hours. Latencies follow a diurnal pattern similar to the request rate and 99.9 percentile latencies are an order of magnitude higher than averages

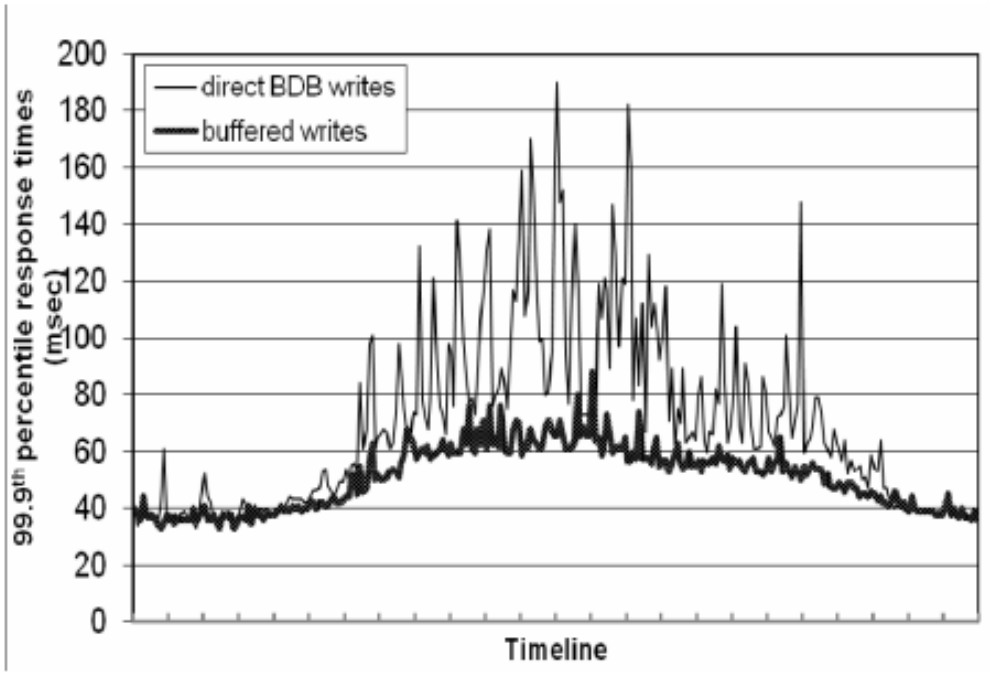


Figure 5: Comparison of performance of 99.9th percentile latencies for buffered vs. non-buffered writes over a period of 24 hours. The intervals between consecutive ticks in the x-axis correspond to one hour.

Experiences and Lessons

- **Ensuring Uniform Load distribution**
- Dynamo's design assumes that even where there is a significant skew in the access distribution there are enough keys in the popular end of the distribution so that the load of handling popular keys can be spread across the nodes uniformly through partitioning.
- The total number of requests received by each node was measured for a period of 24 hours - broken down into intervals of 30 minutes.
- In a given time window, a node is considered to be "inbalance", if the node's request load **deviates from the average load** by a value a less than a certain threshold (here 15%).
- Otherwise the node was deemed "out-of-balance".

Experiences and Lessons

- Figure 6 presents the fraction of nodes that are “out-of-balance” (henceforth, “imbalance ratio”) during this time period.
- During low loads the imbalance ratio is as high as 20% and during high loads it is close to 10%.
- Intuitively, this can be explained by the fact that under high loads, a large number of popular keys are accessed and due to uniform distribution of keys the load is evenly distributed.

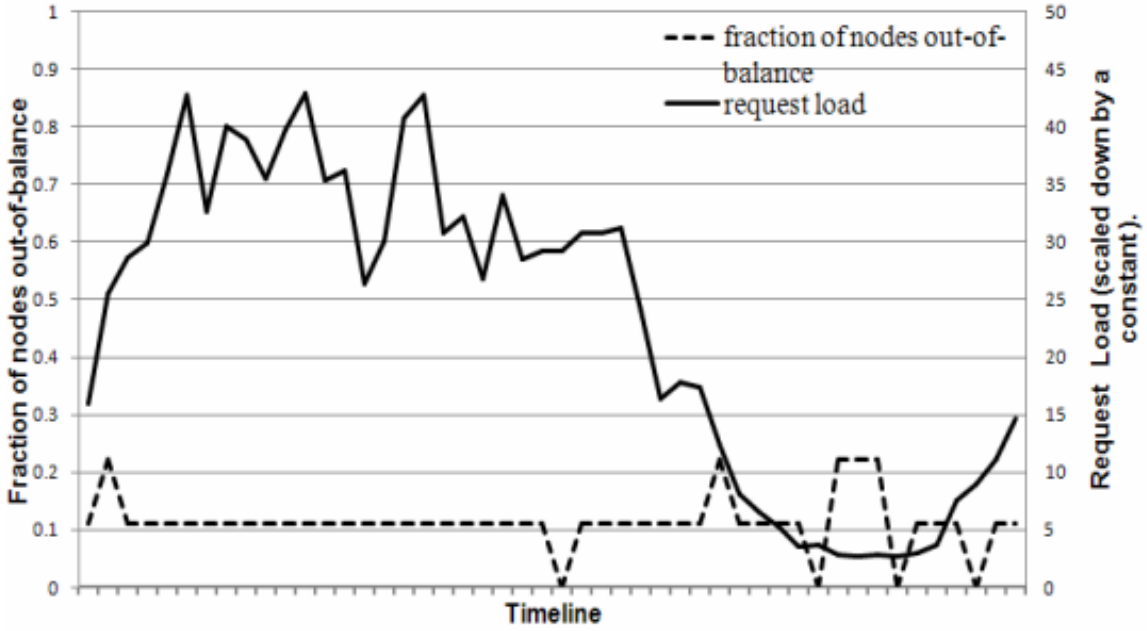


Figure 6: Fraction of nodes that are out-of-balance (i.e., nodes whose request load is above a certain threshold from the average system load) and their corresponding request load. The interval between ticks in x-axis corresponds to a time period of 30 minutes.

Experiences and Lessons

- How Dynamo's partitioning scheme has evolved over time and its implications on load distribution

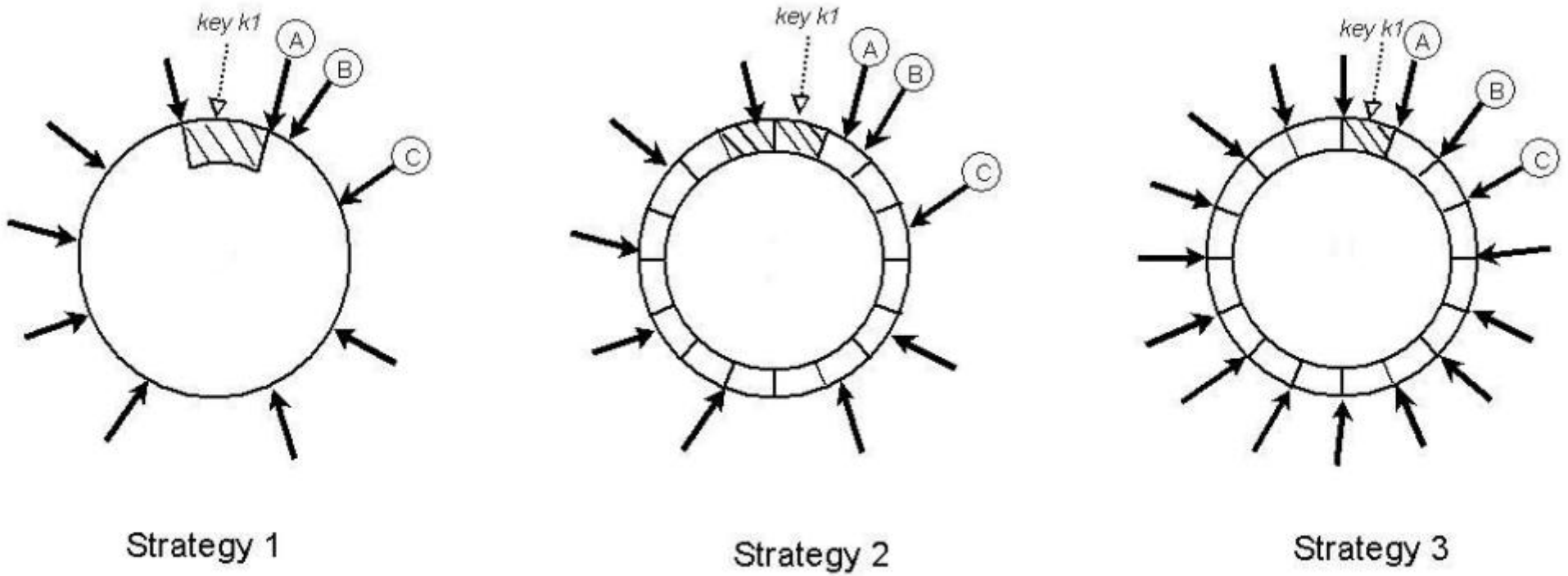
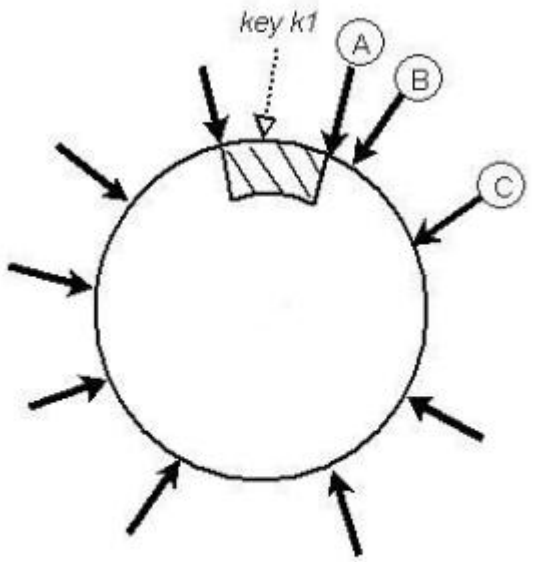


Figure 7: Partitioning and placement of keys in the three strategies. A, B, and C depict the three unique nodes that form the preference list for the key k1 on the consistent hashing ring (N=3). The shaded area indicates the key range for which nodes A, B, and C form the preference list. Dark arrows indicate the token locations for various nodes.

Experiences and Lessons

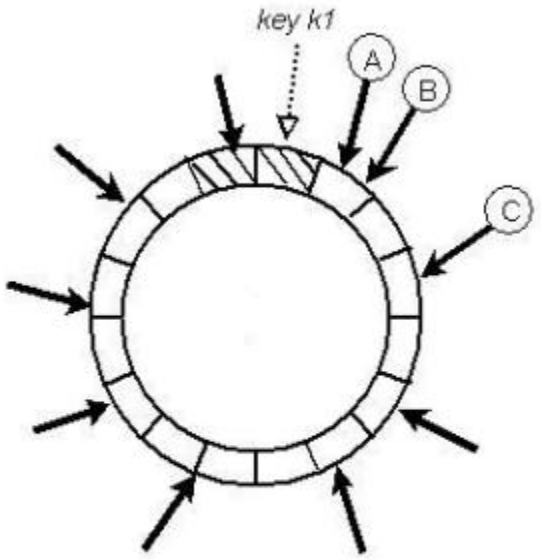
- ***Strategy 1**: T random tokens per node and partition **by token value***:
- Each node is assigned T tokens (chosen uniformly at random from the hash space).
- The last token and the first token form **a range** that "wraps" around from the highest value to the lowest value **in the hash space**.
- Note that the **space needed to maintain the membership** at each node increases **linearly with the number of nodes** in the system.
- The fundamental issue with this strategy is that the schemes for **data partitioning and data placement are intertwined**.
  - For instance, it is preferred to add more nodes to the system in order to handle an increase in request load.
  - However, in this scenario, it is **not possible to add nodes without affecting data partitioning**.



Strategy 1

Experiences and Lessons

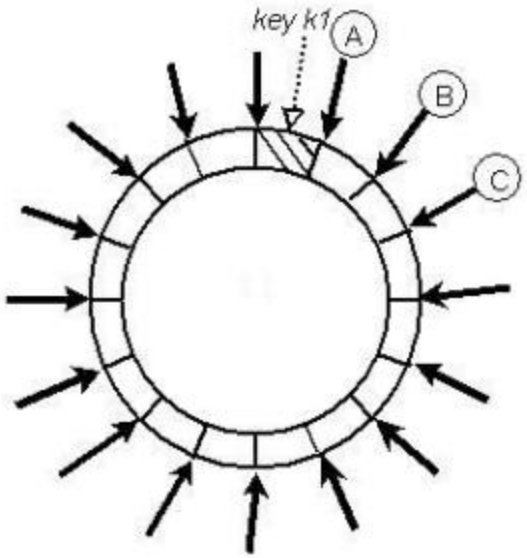
- ***Strategy 2****: T random tokens per node and equal sized partitions:*
- The hash space is divided into Q equally sized partitions/ranges and each node is assigned T random tokens.
- Q is usually set such that Q >> N and Q >> S*T, where S is the number of nodes in the system.
- In this strategy, the tokens are only used to build the function that maps values in the hash space to the ordered lists of nodes and not to decide the partitioning.
- **A hash partition is placed on the first N unique nodes** that are encountered while walking the consistent hashing ring clockwise from the end of the partition.
- The primary **advantages of this strategy** are: (i) decoupling of partitioning and partition placement, and (ii) enabling the possibility of changing the placement scheme at runtime.



Strategy 2

Experiences and Lessons

- ***Strategy 3****: Q/S tokens per node, equal-sized partitions:*
- Divides the hash space into Q equally sized partitions and the placement of partition is decoupled from the partitioning scheme.
- Each node is assigned Q/S tokens where S is the number of nodes in the system.
- When a node leaves the system, its tokens are randomly distributed to the remaining nodes such that these properties are preserved.



Strategy 3

Experiences and Lessons

- The efficiency of these three strategies is evaluated for a system with S=30 and N=3.
- The load distribution property of
  - strategy 1 depends on the number of tokens (i.e., T)
  - while strategy 3 depends on the number of partitions (i.e., Q).
- All strategies use the same amount of space to maintain their membership information.
  - For instance, in strategy 1 each node needs to maintain the **token positions** of all the nodes in the ring.
  - In strategy 3 each node needs to maintain the information regarding the **partitions** assigned to each node.

Experiences and Lessons

- ***Load balancing efficiency*** is defined as the ratio of average number of requests served by each node to the maximum number of requests served by the hottest node.
- Compared to Strategy 1, Strategy 3 achieves better efficiency and reduces the **size of membership information** maintained at each node by three orders of magnitude.

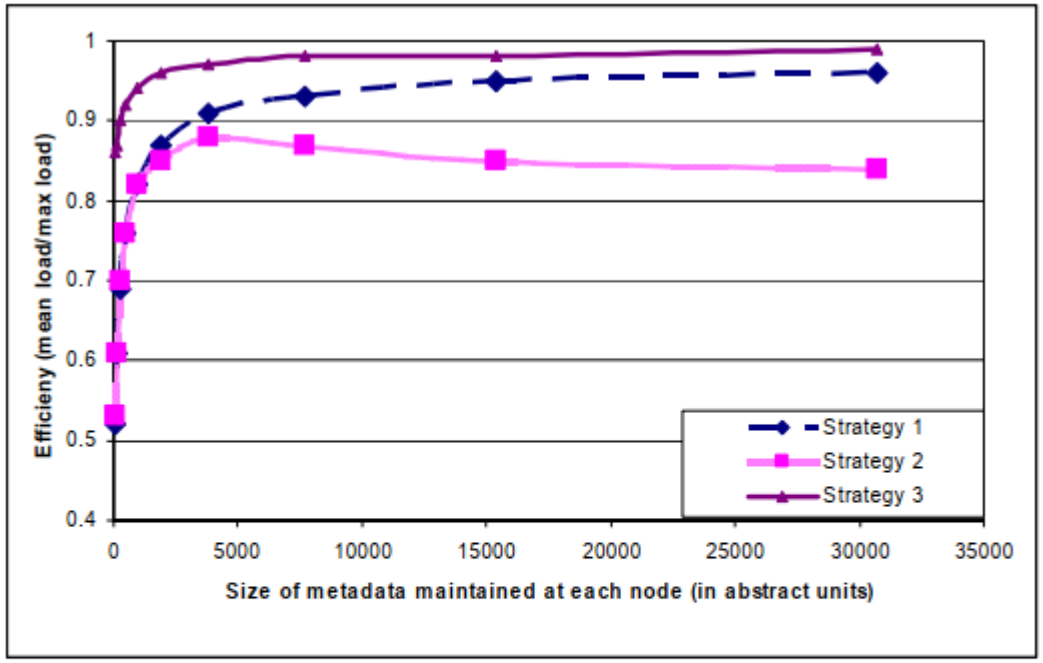


Figure 8: Comparison of the load distribution efficiency of different strategies for system with 30 nodes and N=3 with equal amount of metadata maintained at each node. The values of the system size and number of replicas are based on the typical configuration deployed for majority of our services.

Experiences and Lessons

- Nodes gossip the membership information periodically and as such it is desirable to keep this information as compact as possible.
- The disadvantage of strategy 3 is that changing the node membership requires coordination in order to preserve the properties required of the assignment.

Experiences and Lessons

- **Divergent Versions: When and How Many?**
- The observation is that divergent versions are **created rarely**.
- Experience shows that the increase in the number of divergent versions is contributed **not by failures** but due to the increase in number of **concurrent writers**.
- The increase in the number of concurrent writes is usually triggered by busy robots (automated client programs) and rarely by humans.

Experiences and Lessons

- **Client-driven or Server-driven Coordination**
- An alternative approach to request coordination is to **move the state machine to the client nodes**.
- In this scheme client applications use a library to perform request coordination locally.
- A client periodically picks a random Dynamo node and downloads its current view of Dynamo membership state.
- Using this information **the client can determine which set of nodes form the preference list** for any given key.
- Read requests can be coordinated at the client node thereby **avoiding the extra network hop** that is incurred if the request were assigned to a random Dynamo node by the load balancer.

Experiences and Lessons

Table 2: Performance of client-driven and server-driven coordination approaches.

	99.9th percentile read latency (ms)	99.9th percentile write latency (ms)	Average read latency (ms)	Average write latency (ms)
Server-driven	68.9	68.5	3.9	4.02
Client-driven	30.4	30.4	1.55	1.9

- The latency improvement is because the client-driven approach eliminates the overhead of the load balancer and the extra network hop that may be incurred when a request is assigned to a random node.

Discussion

- Dynamo adopts a full membership model **where each node is aware of the data hosted by its peers**.
- To do this, each node **actively gossips the full routing table with other nodes** in the system.
- This model works well for a system that contains couple of hundreds of nodes.
- However, scaling such a design to run with tens of thousands of nodes is not trivial because the overhead in maintaining the routing table increases with the system size.
- This limitation might be overcome by introducing hierarchical extensions to Dynamo.
- Also, note that this problem is actively addressed by O(1) DHT systems(e.g., [14]).

[14] Ramasubramanian, V., and Sirer, E. G. Beehive: O(1)lookup performance for power-law query distributions in peer-topeer overlays. In *Proceedings of the 1st Conference on Symposium on Networked Systems Design and Implementation,* San Francisco, CA, March 29 - 31, 2004.